

Desafio Técnico – Looqbox

Analista de Dados e BI

Introdução

Este documento apresenta a solução desenvolvida para o desafio técnico proposto pela Looqbox, cujo objetivo é avaliar competências relacionadas à manipulação de dados, consultas SQL, programação em Python e análise exploratória.

O desafio foi dividido em três etapas principais. A primeira consistiu na elaboração de consultas SQL utilizando o esquema disponibilizado para responder perguntas de negócio relacionadas a produtos, departamentos e vendas. Essa etapa teve como objetivo avaliar a capacidade de extração, filtragem, agregação e interpretação de dados em bancos relacionais.

A segunda etapa consistiu na criação de uma função reutilizável para consulta da tabela de vendas (`data_product_sales`), permitindo a aplicação de filtros dinâmicos por produto, loja e período. Essa atividade teve como foco demonstrar boas práticas de programação, reutilização de código e construção dinâmica de consultas SQL.

Por fim, a terceira etapa teve como objetivo explorar a base de filmes (`IMDB_movies`) por meio de consultas e análises utilizando Python e Pandas. A partir dos dados disponíveis, foram realizadas agregações e visualizações para identificar padrões relevantes, como os gêneros e diretores com maiores avaliações médias.

Ao longo do desenvolvimento, foram utilizados SQL para extração e agregação dos dados, Python para implementação das soluções, Pandas para tratamento e análise das informações e Matplotlib para construção das visualizações. A abordagem adotada buscou priorizar clareza, organização do código e capacidade de reutilização, aspectos fundamentais em projetos de análise de dados e Business Intelligence.

Parte 1 – Teste SQL

Objetivo

Nesta etapa, foram desenvolvidas consultas SQL utilizando o esquema lookbox_challenge com o objetivo de responder às questões de negócio propostas no desafio. As consultas foram executadas em um banco MySQL utilizando Python, Pandas e PyMySQL para conexão e visualização dos resultados.

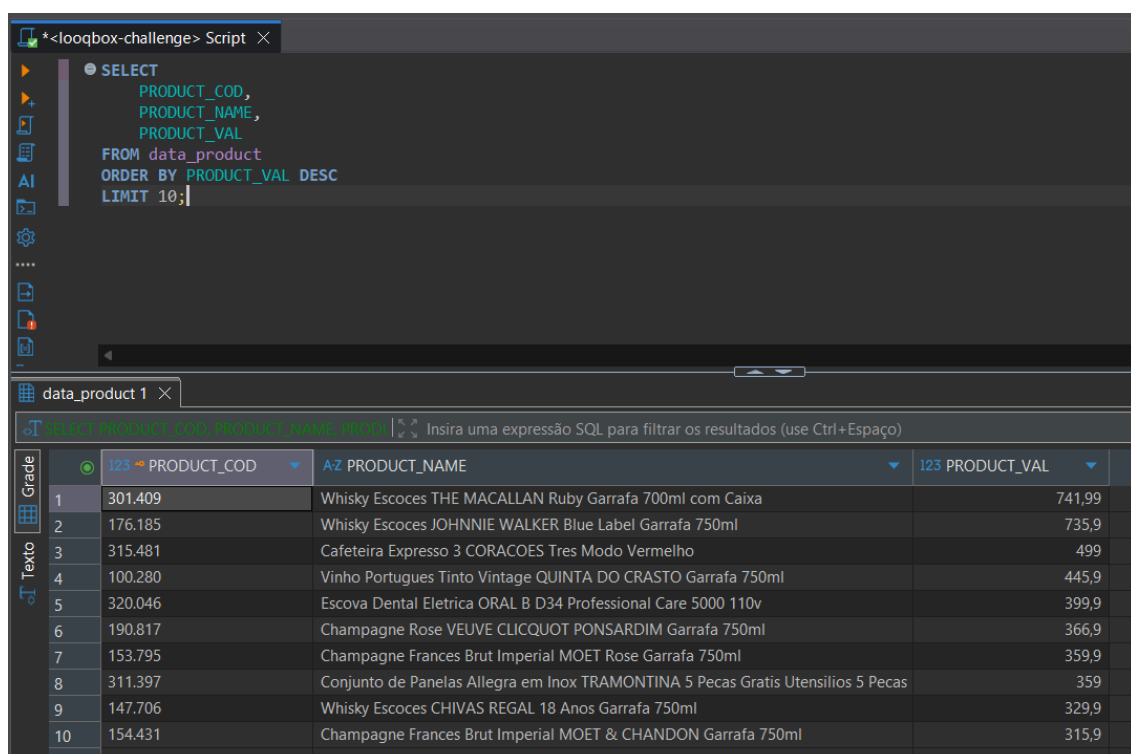
Ferramentas Utilizadas

Ferramenta	Utilização
MySQL	Armazenamento dos dados
SQL	Extração e agregação das informações
Python	Execução das consultas
Pandas	Manipulação e visualização dos resultados
PyMySQL	Conexão com o banco de dados

Questão 1 – Quais são os 10 produtos mais caros da empresa?

Estratégia

Para responder à pergunta, foi necessário consultar a tabela de produtos e ordenar os registros pelo valor do produto em ordem decrescente, retornando apenas os 10 primeiros registros.



The screenshot shows a SQL IDE interface. At the top, a script editor contains the following SQL query:

```
SELECT
    PRODUCT_COD,
    PRODUCT_NAME,
    PRODUCT_VAL
FROM data_product
ORDER BY PRODUCT_VAL DESC
LIMIT 10;
```

Below the script editor, a table viewer displays the results of the query. The table has four columns: a row number, PRODUCT_COD, PRODUCT_NAME, and PRODUCT_VAL. The results are sorted by PRODUCT_VAL in descending order.

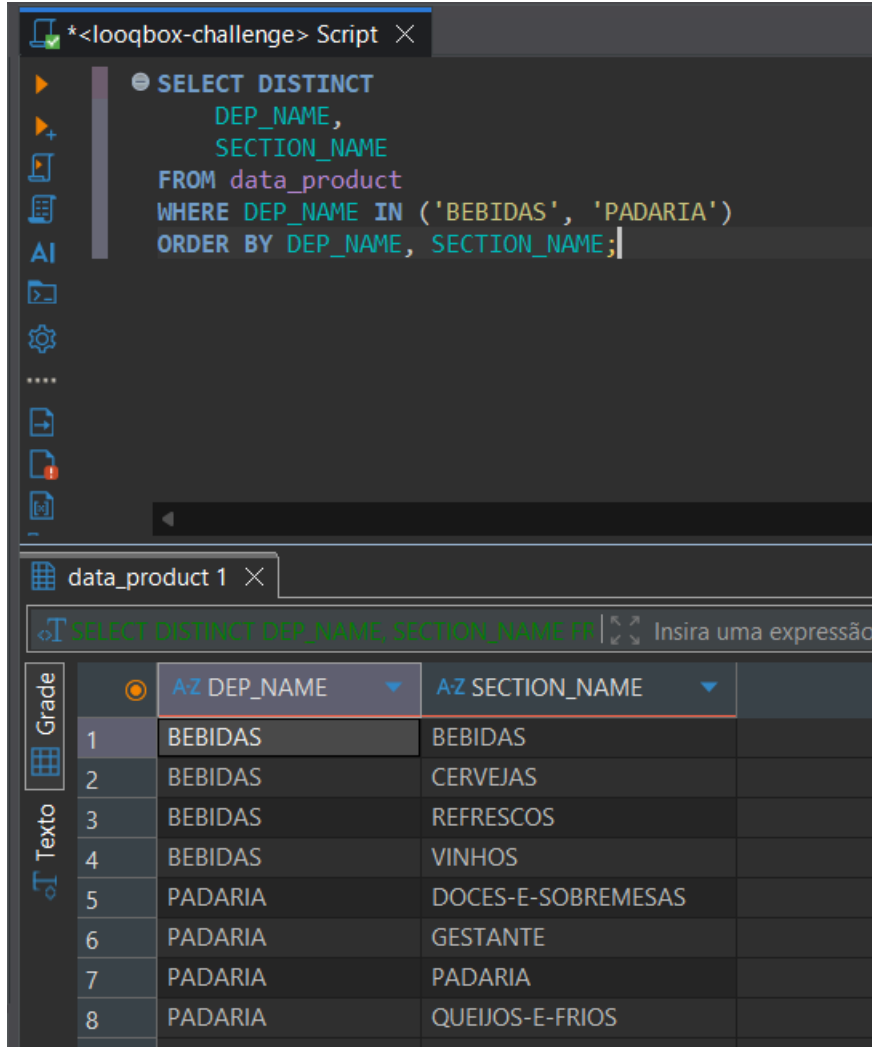
	123 PRODUCT_COD	AZ PRODUCT_NAME	123 PRODUCT_VAL
1	301.409	Whisky Escoces THE MACALLAN Ruby Garrafa 700ml com Caixa	741,99
2	176.185	Whisky Escoces JOHNNIE WALKER Blue Label Garrafa 750ml	735,9
3	315.481	Cafeteira Expresso 3 CORACOES Tres Modo Vermelho	499
4	100.280	Vinho Portugues Tinto Vintage QUINTA DO CRASTO Garrafa 750ml	445,9
5	320.046	Escova Dental Eletrica ORAL B D34 Professional Care 5000 110v	399,9
6	190.817	Champagne Rose VEUVE CLICQUOT PONSARDIM Garrafa 750ml	366,9
7	153.795	Champagne Frances Brut Imperial MOET Rose Garrafa 750ml	359,9
8	311.397	Conjunto de Panelas Allegra em Inox TRAMONTINA 5 Pecas Gratis Utensilios 5 Pecas	359
9	147.706	Whisky Escoces CHIVAS REGAL 18 Anos Garrafa 750ml	329,9
10	154.431	Champagne Frances Brut Imperial MOET & CHANDON Garrafa 750ml	315,9

PRODUCT_COD	PRODUCT_NAME	PRODUCT_VAL
301409	Whisky Escoces THE MACALLAN Ruby Garrafa 700ml com Caixa	741.99
176185	Whisky Escoces JOHNNIE WALKER Blue Label Garrafa 750ml	735.90
315481	Cafeteira Expresso 3 CORACOES Tres Modo Vermelho	499.00
100280	Vinho Portugues Tinto Vintage QUINTA DO CRASTO Garrafa 750ml	445.90
320046	Escova Dental Eletrica ORAL B D34 Professional Care 5000 110v	399.90
190817	Champagne Rose VEUVE CLICQUOT PONSARDIM Garrafa 750ml	366.90
153795	Champagne Frances Brut Imperial MOET Rose Garrafa 750ml	359.90
311397	Conjunto de Panelas Allegra em Inox TRAMONTINA 5 Pecas Gratis Utensilios 5 Pecas	359.00
147706	Whisky Escoces CHIVAS REGAL 18 Anos Garrafa 750ml	329.90
154431	Champagne Frances Brut Imperial MOET & CHANDON Garrafa 750ml	315.90

Questão 2 – Quais são as seções dos departamentos BEBIDAS e PADARIA?

Estratégia

O objetivo foi identificar todas as seções associadas aos departamentos solicitados, removendo possíveis duplicidades.



The screenshot shows a SQL script editor with the following query:

```
SELECT DISTINCT
  DEP_NAME,
  SECTION_NAME
FROM data_product
WHERE DEP_NAME IN ('BEBIDAS', 'PADARIA')
ORDER BY DEP_NAME, SECTION_NAME;
```

Below the script, the results are displayed in a table. The table has two columns: 'DEP_NAME' and 'SECTION_NAME'. The results are as follows:

Grade	DEP_NAME	SECTION_NAME
1	BEBIDAS	BEBIDAS
2	BEBIDAS	CERVEJAS
3	BEBIDAS	REFRESCOS
4	BEBIDAS	VINHOS
5	PADARIA	DOCES-E-SOBREMESAS
6	PADARIA	GESTANTE
7	PADARIA	PADARIA
8	PADARIA	QUEIJOS-E-FRIOS

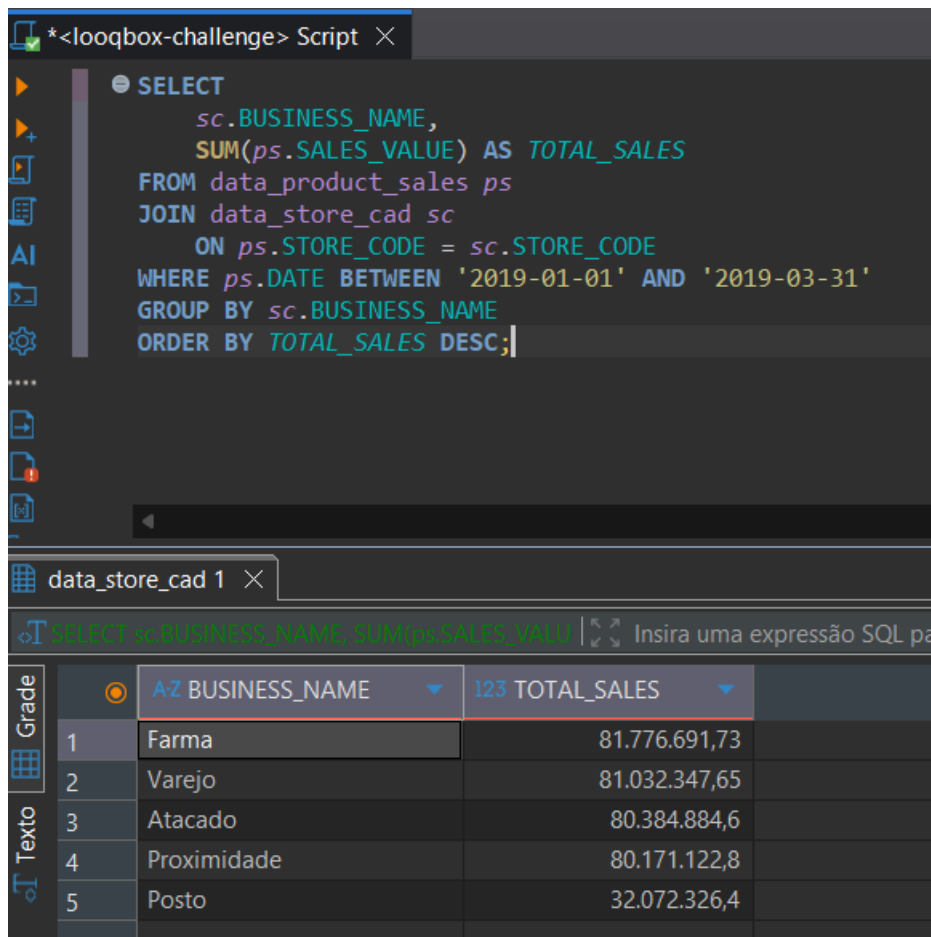
Explicação

- WHERE DEP_NAME IN (...) filtra apenas os departamentos desejados.
- DISTINCT elimina registros duplicados.
- ORDER BY organiza a visualização dos resultados.

Questão 3 – Qual foi o total de vendas por área de negócio no primeiro trimestre de 2019?

Estratégia

Foi necessário relacionar a tabela de vendas com a tabela de cadastro das lojas para identificar a área de negócio associada a cada venda.



The screenshot shows a SQL IDE window titled '*<looqbox-challenge> Script'. The query editor contains the following SQL code:

```
SELECT
    sc.BUSINESS_NAME,
    SUM(ps.SALES_VALUE) AS TOTAL_SALES
FROM data_product_sales ps
JOIN data_store_cad sc
    ON ps.STORE_CODE = sc.STORE_CODE
WHERE ps.DATE BETWEEN '2019-01-01' AND '2019-03-31'
GROUP BY sc.BUSINESS_NAME
ORDER BY TOTAL_SALES DESC;
```

Below the query editor, a table viewer shows the results of the query. The table has two columns: 'BUSINESS_NAME' and 'TOTAL_SALES'. The results are sorted in descending order of total sales.

Grade	BUSINESS_NAME	TOTAL_SALES
1	Farma	81.776.691,73
2	Varejo	81.032.347,65
3	Atacado	80.384.884,6
4	Proximidade	80.171.122,8
5	Posto	32.072.326,4

Explicação

- JOIN relaciona vendas e cadastro das lojas.
- SUM(SALES_VALUE) calcula o total vendido.
- BETWEEN restringe ao primeiro trimestre de 2019.
- GROUP BY agrupa por área de negócio.
- ORDER BY apresenta os resultados da maior para a menor receita.

Parte 2 do desafio

Caso 1 – Desenvolvimento de Função Dinâmica para Consulta de Vendas

O primeiro caso proposto teve como objetivo o desenvolvimento de uma função em Python para realizar consultas dinâmicas na tabela `data_product_sales`.

A proposta surgiu da necessidade de reduzir a repetição de consultas semelhantes, que diferiam apenas pelos critérios de filtragem aplicados. Para solucionar esse problema, foi criada uma função reutilizável capaz de construir consultas SQL de forma flexível, permitindo filtrar os dados por produto, loja e intervalo de datas.

Além de retornar os resultados diretamente em um `DataFrame` do Pandas, a solução foi projetada com foco em reutilização, legibilidade e facilidade de manutenção. Considerando que a função poderia ser utilizada por diferentes equipes, também foram adotadas práticas de documentação e organização do código, tornando sua utilização mais intuitiva e escalável.

Para o desenvolvimento desta solução foram utilizadas as seguintes tecnologias:

- Python para implementação da lógica da função
- SQL para construção dinâmica das consultas
- Pandas para leitura dos dados e retorno em formato `DataFrame`
- PyMySQL para conexão com o banco de dados MySQL disponibilizado no desafio
- Google Colab como ambiente de desenvolvimento

A estratégia adotada foi criar uma única função capaz de construir a cláusula `WHERE` dinamicamente de acordo com os parâmetros informados pelo usuário.

Caso nenhum parâmetro seja fornecido, a função retorna todos os registros da tabela. À medida que filtros são adicionados, novas condições são incorporadas à consulta SQL.

Essa abordagem reduz duplicação de código e facilita futuras manutenções e expansões.

Implementação:

```
[19]
✓ 0s ▶ def retrieve_data(product_code=None, store_code=None, date=None):
      """
      Recupera dados da tabela data_product_sales.

      Parameters
      -----
      product_code : int
          Código do produto.

      store_code : int
          Código da loja.

      date : str ou list
          Data única:
          '2019-01-01'

          ou intervalo:
          ['2019-01-01', '2019-01-31']

      Returns
      -----
      pandas.DataFrame
      """

      query = """
      SELECT
          STORE_CODE,
          PRODUCT_CODE,
          DATE,
          SALES_VALUE,
          SALES_QTY
      FROM data_product_sales
      WHERE 1=1
      """

      if product_code is not None:
          query += f" AND PRODUCT_CODE = {product_code}"

      if store_code is not None:
          query += f" AND STORE_CODE = {store_code}"

      if date is not None:

          if isinstance(date, list) and len(date) == 2:
              query += f" AND DATE BETWEEN '{date[0]}' AND '{date[1]}'"

          elif isinstance(date, str):
              query += f" AND DATE = '{date}'"

      print("SQL gerado:")
      print(query)

      df = pd.read_sql(query, conn)

      return df
```


Exemplo de Utilização:

```
[20] my_data = retrieve_data(  
✓ Os   product_code=18,  
       store_code=1,  
       date=['2019-01-01', '2019-01-31']  
       )  
  
my_data.head()
```

SQL gerado:

```
SELECT  
    STORE_CODE,  
    PRODUCT_CODE,  
    DATE,  
    SALES_VALUE,  
    SALES_QTY  
FROM data_product_sales  
WHERE 1=1  
      AND PRODUCT_CODE = 18 AND STORE_CODE = 1 AND DATE BETWEEN '2019-01-01' AND '2019-01-31'
```

/tmp/ipykernel_4179/2462962621.py:53: UserWarning: pandas only supports SQLAlchemy connect
df = pd.read_sql(query, conn)

	STORE_CODE	PRODUCT_CODE	DATE	SALES_VALUE	SALES_QTY
0	1	18	2019-01-01	708.5	65.0
1	1	18	2019-01-02	1297.1	119.0
2	1	18	2019-01-03	1144.5	105.0
3	1	18	2019-01-04	1090.0	100.0
4	1	18	2019-01-05	893.8	82.0

A função retornou corretamente os registros correspondentes ao produto 18, loja 1 e período entre 01/01/2019 e 31/01/2019.

O resultado foi disponibilizado em formato DataFrame, permitindo análises posteriores utilizando Pandas ou bibliotecas de visualização.

Possíveis melhorias :

Embora a solução atenda aos requisitos do desafio, algumas melhorias poderiam ser implementadas em um ambiente produtivo:

- Utilização de consultas parametrizadas para aumentar a segurança contra SQL Injection
- Validação de tipos de entrada
- Tratamento de exceções de conexão e execução
- Utilização de SQLAlchemy para gerenciamento da conexão
- Inclusão de logs para monitoramento da execução

Caso 2 – Construção de Visualização a partir de Consultas Pré-definidas

No segundo caso, o cliente forneceu duas consultas SQL previamente definidas e solicitou a construção de uma visualização em Python sem que as consultas fossem modificadas.

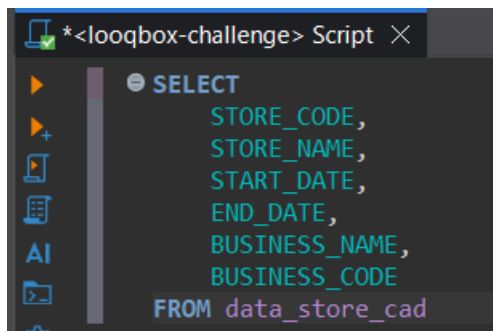
Além disso, foi solicitado que os dados fossem considerados apenas para o período compreendido entre 01/10/2019 e 31/12/2019. A partir das informações retornadas pelas consultas, foi necessário combinar os dados, realizar os cálculos necessários e reproduzir a visualização solicitada pelo cliente.

O principal desafio desta etapa consistiu em integrar diferentes fontes de informação, aplicar filtros temporais e gerar indicadores de negócio capazes de alimentar a visualização final.

Ferramentas Utilizadas

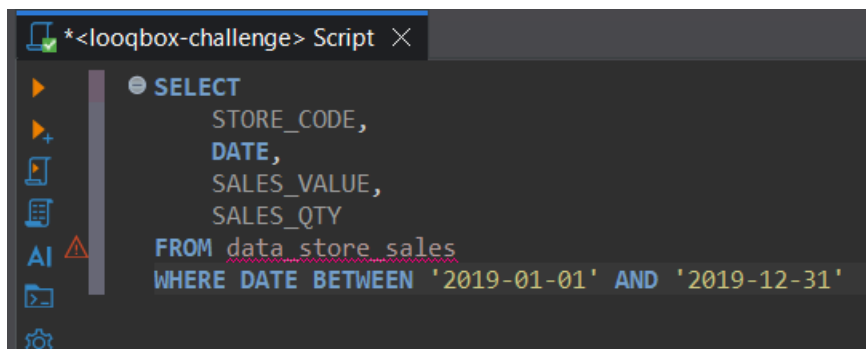
- SQL
- Python
- Pandas
- Matplotlib
- PyMySQL
- Google Colab

Consulta 1:

A screenshot of a SQL query editor window titled '*<looqbox-challenge> Script'. The editor shows a SQL query:

```
SELECT
    STORE_CODE,
    STORE_NAME,
    START_DATE,
    END_DATE,
    BUSINESS_NAME,
    BUSINESS_CODE
FROM data_store_cad
```

Consulta 2:

A screenshot of a SQL query editor window titled '*<looqbox-challenge> Script'. The editor shows a SQL query:

```
SELECT
    STORE_CODE,
    DATE,
    SALES_VALUE,
    SALES_QTY
FROM data_store_sales
WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'
```

A primeira consulta foi utilizada para obter os dados cadastrais das lojas, incluindo nome, período de operação e categoria de negócio. Já a segunda consulta retornou os registros de vendas realizados ao longo de 2019, contendo o valor e a quantidade vendida por loja. Essas informações foram posteriormente integradas para o cálculo do Ticket Médio (TM) e construção da visualização solicitada pelo cliente.



Observação: Devido às limitações de espaço do documento PDF, a visualização apresentada foi redimensionada para melhor adequação ao layout. A versão completa, com melhor resolução, detalhes visuais e experiência de análise, pode ser consultada diretamente no notebook `Looqbox_Challenge_Final_Gabriel_Wesley.ipynb`, onde o dashboard foi desenvolvido originalmente em Python.

Para a construção da visualização foram utilizadas as seguintes ferramentas:

- Python: utilizado como linguagem principal para manipulação dos dados e construção da visualização.
- Pandas: utilizado para estruturar os dados em formato tabular, ordenar os registros e calcular os indicadores necessários.
- NumPy: utilizado na criação dos gradientes de fundo e das barras do gráfico, permitindo maior controle visual.
- Matplotlib: utilizado para construção da visualização final em formato estático, adequado para inserção no relatório em PDF.
- FancyBboxPatch: recurso do Matplotlib utilizado para criar elementos visuais com cantos arredondados, como os cards de KPI e as barras do gráfico.

Sobre o código utilizado para realização do gráfico:

```
tm_data = pd.DataFrame({
    "Loja": [
        "Bahia", "Bangkok", "Belém", "Berlim", "Buenos Aires",
        "Chicago", "Dubai", "Hong Kong", "Londres", "Madri",
        "Miami", "Nova Iorque", "Paris", "Rio de Janeiro", "Roma",
        "Salvador", "São Paulo", "Sidney", "Tóquio", "Vancouver"
    ],
    "Categoria": [
        "Atacadinho", "Posto", "Proximidade", "Proximidade", "Atacadinho",
        "Varejo", "Atacadinho", "Farma", "Farma", "Farma",
        "Posto", "Proximidade", "Proximidade", "Farma", "Varejo",
        "Atacadinho", "Varejo", "Posto", "Varejo", "Posto"
    ],
    "TM": [
        15.39, 13.67, 15.37, 15.39, 15.39,
        15.53, 15.39, 26.35, 28.99, 29.03,
        13.67, 15.39, 15.39, 29.59, 15.39,
        15.39, 15.39, 13.67, 15.39, 13.67
    ]
})

df = tm_data.sort_values("TM", ascending=True).reset_index(drop=True)
```

Neste trecho, os dados foram organizados em um DataFrame contendo as informações de loja, categoria e Ticket Médio (TM).

Em seguida, os registros foram ordenados pelo valor do TM para facilitar a visualização e comparação dos resultados no gráfico. Dessa forma, fica mais fácil identificar quais lojas possuem os menores e maiores valores de Ticket Médio.

```
# KPIs
tm_geral = df["TM"].mean()
melhor_loja = df.loc[df["TM"].idxmax(), "Loja"]
melhor_tm = df["TM"].max()
melhor_categoria = df.loc[df["TM"].idxmax(), "Categoria"]

kpis = [
    ("TM Geral", f"{tm_geral:.2f}".replace(".", ",")),
    ("Maior TM", f"{melhor_tm:.2f}".replace(".", ",")),
    ("Destaque", melhor_loja),
    ("Categoria", melhor_categoria)
]
```

Neste trecho, foram calculados alguns indicadores para complementar a análise. Foram obtidos o Ticket Médio geral, a loja com o maior TM, o maior valor encontrado e a categoria associada a esse resultado.

Esses indicadores foram utilizados na criação dos cards de destaque exibidos no topo do dashboard, permitindo uma leitura rápida das principais informações antes da análise detalhada do gráfico.

```
[17]
✓ 5s
def hex_to_rgb(hex_color):
    hex_color = hex_color.replace("#", "")
    return np.array([int(hex_color[i:i+2], 16) for i in (0, 2, 4)] / 255)

def rounded_gradient_barh(ax, y, width, height, color_start, color_end, x_start=0, radius=0.18, zorder=3):
    grad = np.linspace(0, 1, 256).reshape(256, 1)
    c1 = hex_to_rgb(color_start)
    c2 = hex_to_rgb(color_end)
    img = c1 * (1 - grad) + c2 * grad

    patch = FancyBboxPatch(
        (x_start, y - height / 2),
        width,
        height,
        boxstyle=f"round,pad=0,rounding_size={radius}",
        linewidth=0,
        facecolor="none"
    )

    ax.add_patch(patch)

    im = ax.imshow(
        img,
        extent=[x_start, x_start + width, y - height / 2, y + height / 2],
        aspect="auto",
        origin="lower",
        zorder=zorder
    )

    im.set_clip_path(patch)

    fig, ax = plt.subplots(figsize=(11, 6.2), dpi=180)

    fig.patch.set_facecolor("#0F1A24")
    ax.set_facecolor("#0F1A24")

    # Fundo gradiente
    w, h = 1600, 900
    x = np.linspace(0, 1, w)
    y = np.linspace(0, 1, h)
    X, Y = np.meshgrid(x, y)
    diag = (X + Y) / 2

    bg_start = hex_to_rgb("#0F1A24")
    bg_end = hex_to_rgb("#182532")
```

Neste trecho, foram criadas funções responsáveis pela parte visual do dashboard. A função `hex_to_rgb()` converte cores do formato hexadecimal para RGB, permitindo a criação dos efeitos de gradiente utilizados no gráfico.

Já a função `rounded_gradient_barh()` foi desenvolvida para desenhar barras horizontais com cantos arredondados e preenchimento em gradiente, tornando a visualização mais moderna e alinhada à identidade visual escolhida para o projeto.

Também foi definida a estrutura inicial da figura, incluindo tamanho, resolução e cores de fundo. Para reforçar o aspecto visual do dashboard, foi aplicado um gradiente de fundo utilizando tons escuros inspirados na identidade da Looqbox.

```
[17] 5s  # Barras
bar_height = 0.46
max_value = df["TM"].max()

for i, row in df.iterrows():
    is_max = row["TM"] == max_value

    if is_max:
        c1, c2 = "#FFFFFF", "#DCE6EF"
    else:
        c1, c2 = "#64F04A", "#B8FF9A"

    if not is_max:
        ax.barh(
            i,
            row["TM"],
            height=bar_height + 0.10,
            color="#64F04A",
            alpha=0.07,
            zorder=1
        )

    rounded_gradient_barh(
        ax=ax,
        y=i,
        width=row["TM"],
        height=bar_height,
        color_start=c1,
        color_end=c2,
        radius=0.18,
        zorder=3
    )
```

Neste trecho, foi criada a lógica responsável por desenhar as barras do gráfico. Para cada loja, o valor do Ticket Médio é utilizado como tamanho da barra.

A loja com o maior TM recebe um destaque visual utilizando tons claros, enquanto as demais utilizam tons de verde. Essa diferenciação ajuda a identificar rapidamente o melhor resultado da análise.

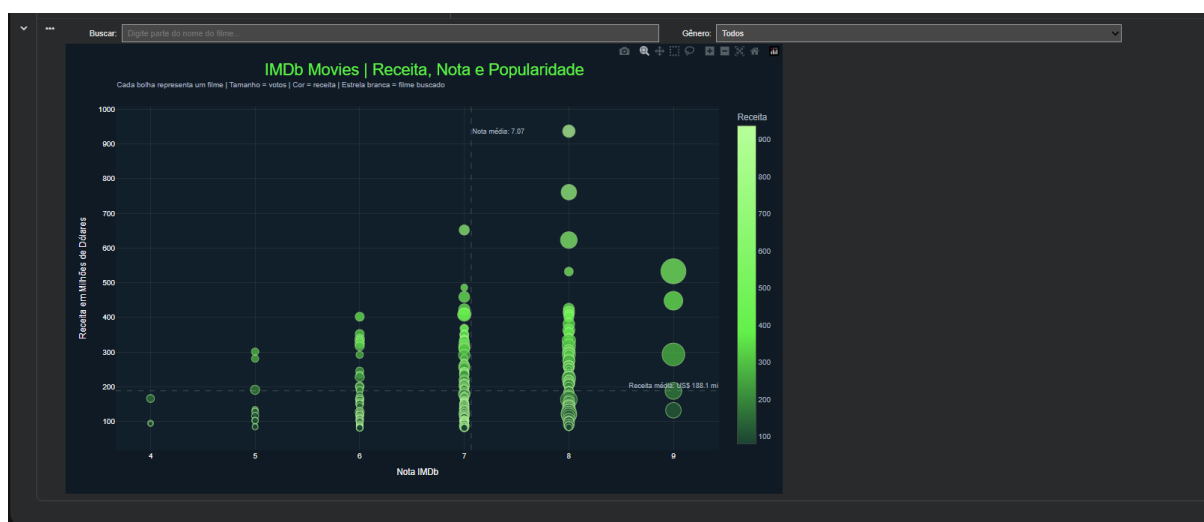
Além disso, foi aplicado um efeito de gradiente e uma leve sombra nas barras para tornar a visualização mais agradável e facilitar a interpretação dos dados.

Parte 3 - Criando sua própria visualização

Para esta etapa, optei por utilizar um gráfico de dispersão com bolhas, pois ele permite analisar diferentes informações ao mesmo tempo de forma visual e intuitiva. Cada bolha representa um filme, sua posição mostra a relação entre nota IMDb e receita gerada, enquanto o tamanho da bolha representa a quantidade de votos recebidos.

Esse formato facilita a identificação de padrões e insights importantes. A partir da visualização, é possível observar que filmes com maior receita nem sempre possuem as maiores notas IMDb, mostrando que sucesso financeiro e avaliação do público não necessariamente seguem o mesmo comportamento. As bolhas maiores ajudam a destacar os filmes mais populares, enquanto as linhas de referência permitem separar os dados em diferentes grupos de análise, como filmes com alta nota e alta receita ou filmes bem avaliados com menor retorno financeiro.

Também foram adicionados recursos de busca e filtros para tornar a exploração dos dados mais simples, permitindo localizar rapidamente um filme específico e analisar seu comportamento em relação aos demais. Para a implementação dessas funcionalidades interativas, utilizei apoio de Inteligência Artificial devido à dificuldade encontrada nessa etapa, enquanto a definição da análise, escolha dos indicadores e construção da visualização permaneceram sob minha responsabilidade.



Devido às limitações de espaço do PDF, a visualização apresentada pode não refletir toda a experiência de navegação e interação disponível no gráfico.

Para uma melhor visualização, incluindo filtros, busca de filmes e interatividade completa, recomenda-se acessar o notebook disponibilizado no Google Colab:

Looqbox_Challenge_Final_Gabriel_Wesley.ipynb

Nele é possível explorar o gráfico de forma mais confortável e visualizar todos os recursos implementados.